

Transformers and Generative AI

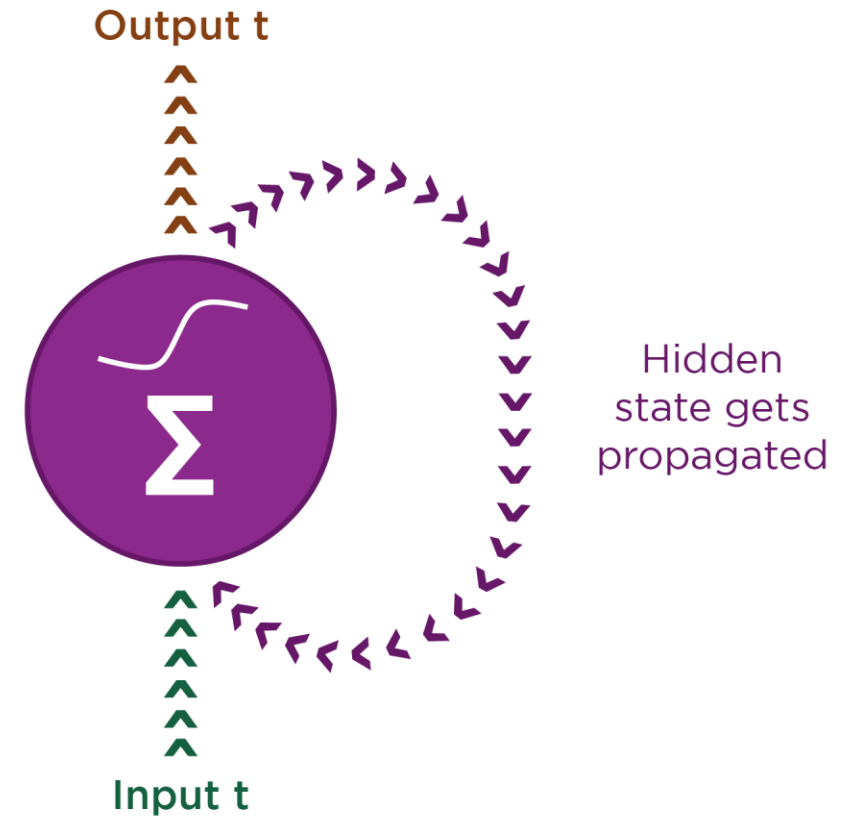
Large language models, transformers, GPT, and AWS support

The Transformer Architecture

And how self-attention works its magic

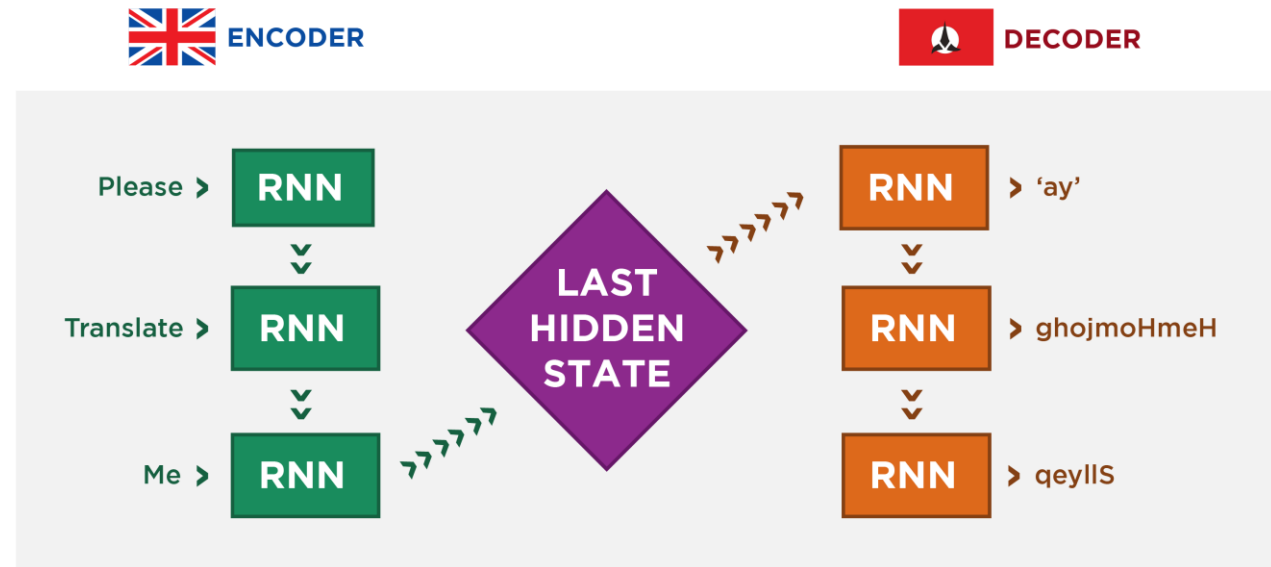
The Evolution of Transformers

- RNN's, LSTMs
- Introduced a feedback loop for propagating information forward
- Useful for modeling sequential things
 - Time series
 - Language! A sequence of words (or tokens)



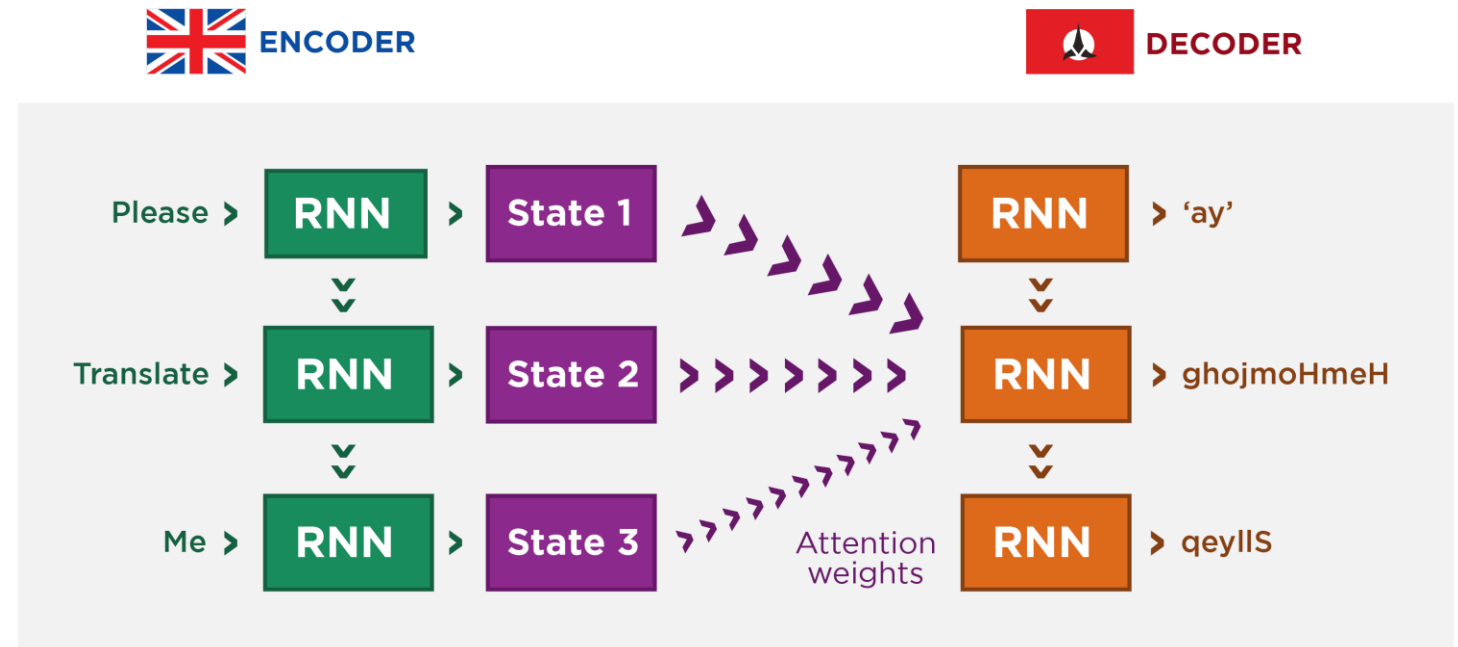
The Evolution of Transformers

- Machine translation
- Encoder / Decoder architecture
- Encoders and Decoders are RNN's
- But, the one vector tying them together creates an information bottleneck
 - Information from the start of the sequence may be lost



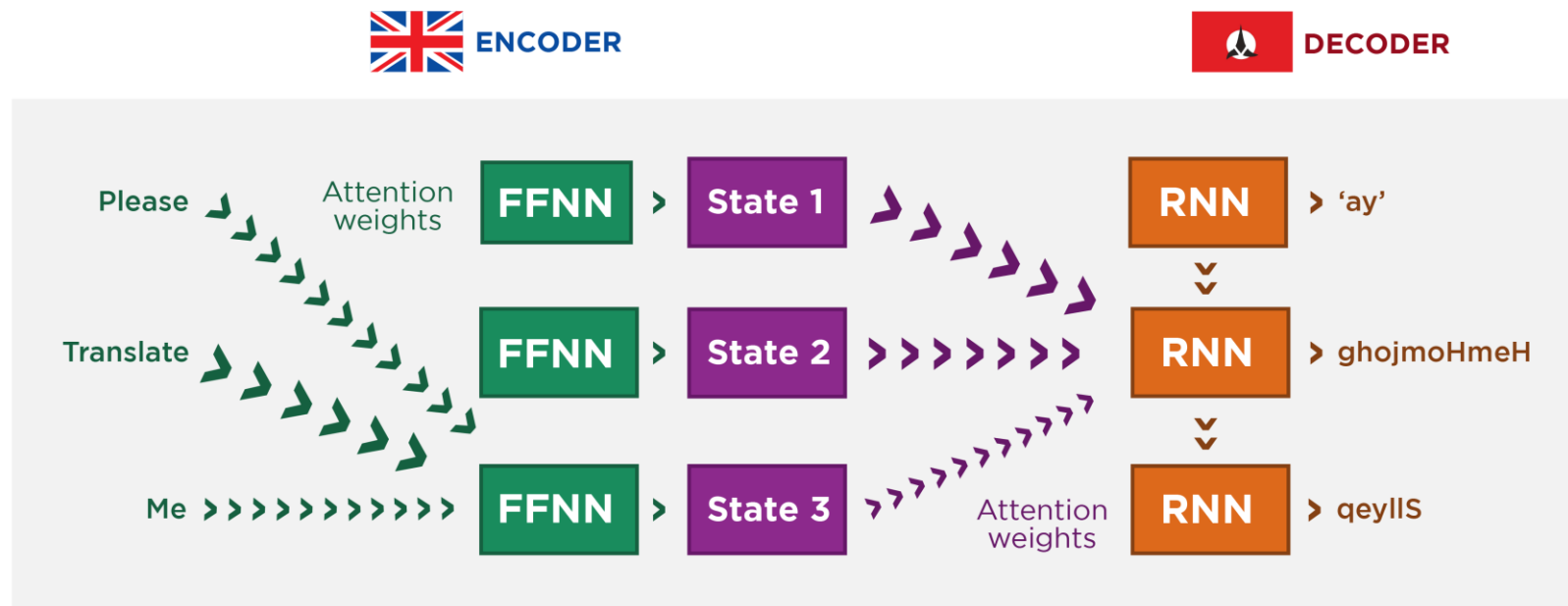
“Attention is all you need”

- A hidden state for each step (token)
- Deals better with differences in word order
- Starts to have a concept of relationships between words
- But RNN's are still sequential in nature, can't parallelize it



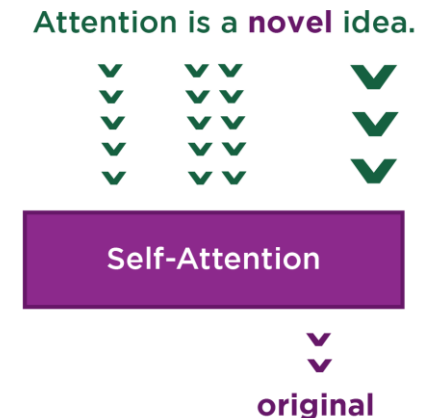
Transformers

- Ditch RNN's for feed-forward neural networks (FFNN's)
- Use “self-attention”
- This makes it parallelizable (so can train on much more data)



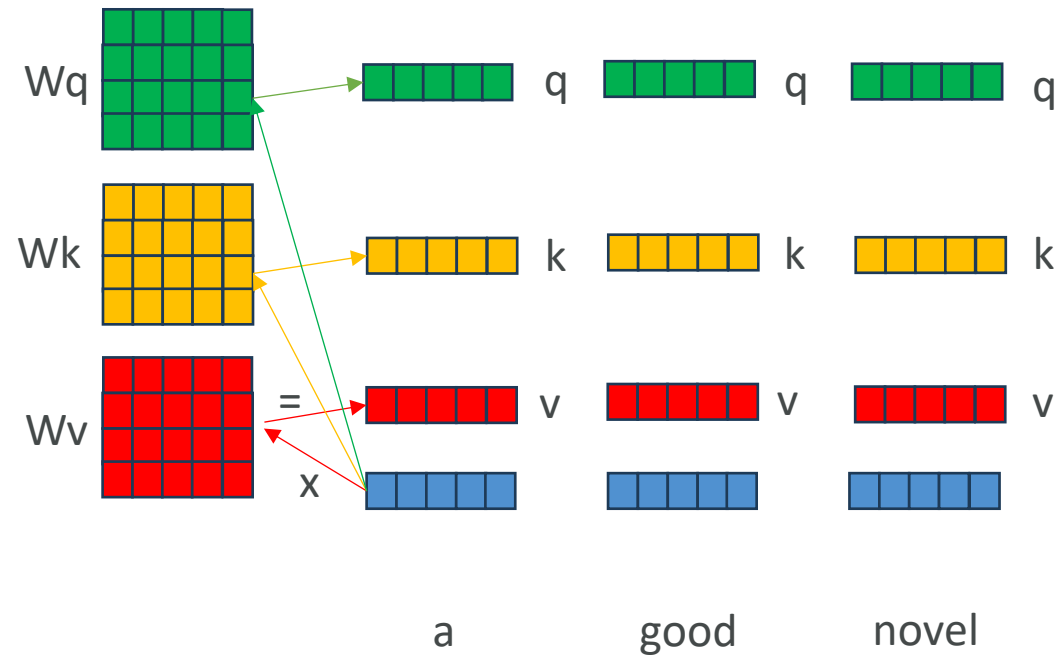
Self-Attention (in more depth)

- Each encoder or decoder has a list of embeddings (vectors) for each token
- Self-attention produces a *weighted average* of all token embeddings. The magic is in computing the attention weights.
- This results in tokens being tied to other tokens that are important for its context, and a new embedding that captures its “meaning” in context.



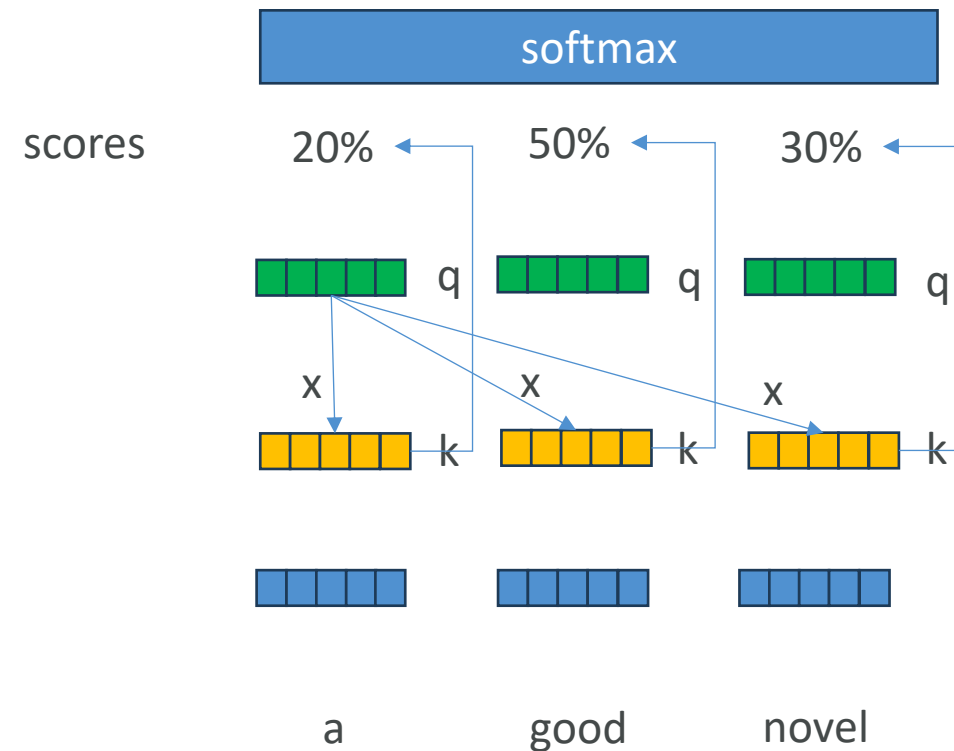
Self-Attention (in more depth)

- Three matrices of weights are learned through back-propagation
 - Query (W_q)
 - Key (W_k)
 - Value (W_v)
- Every token gets a query (q), key (k), and value (v) vector by multiplying its embedding against these matrices



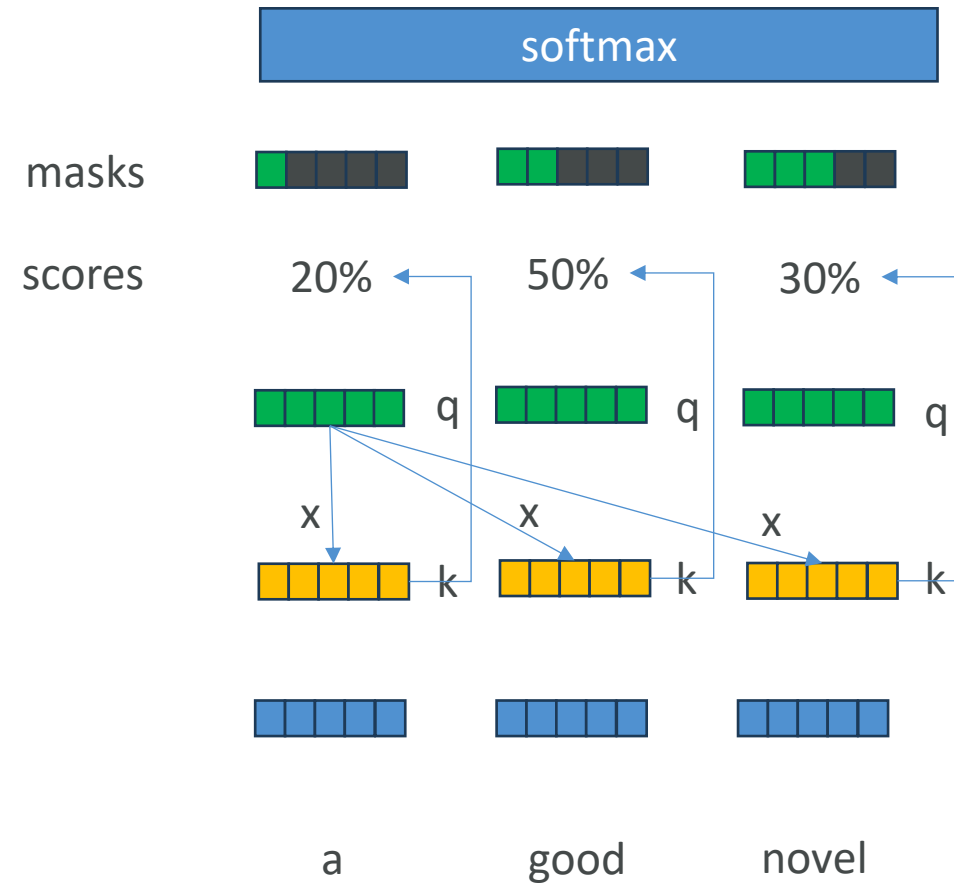
Self-Attention (in more depth)

- Compute a *score* for each token by multiplying (dot product) its query with each key
- “Scaled dot-product attention”
- Dot product is just one *similarity function* we can use.
- In practice, softmax is then applied to the scores to normalize them.



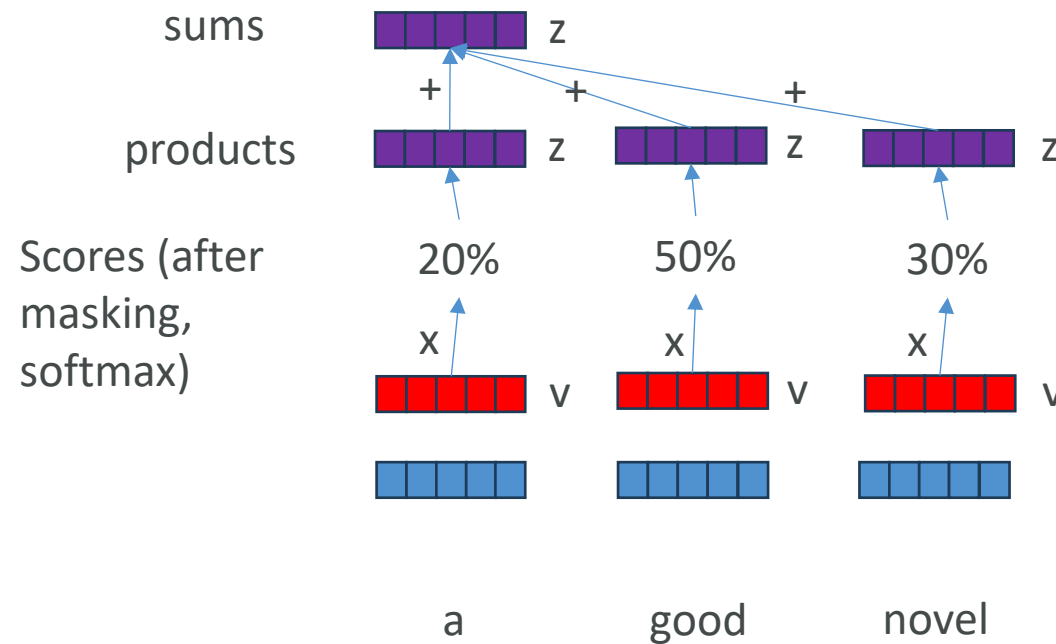
Masked Self-Attention

- A mask can be applied to prevent tokens from “peeking” into future tokens (words)
- GPT does this, but BERT does something else (masked language modeling)
- In this example, “good” wouldn’t be affected by “novel”, but “novel” could be affected by “good”.
- This is just the concept... actual implementation details will vary.



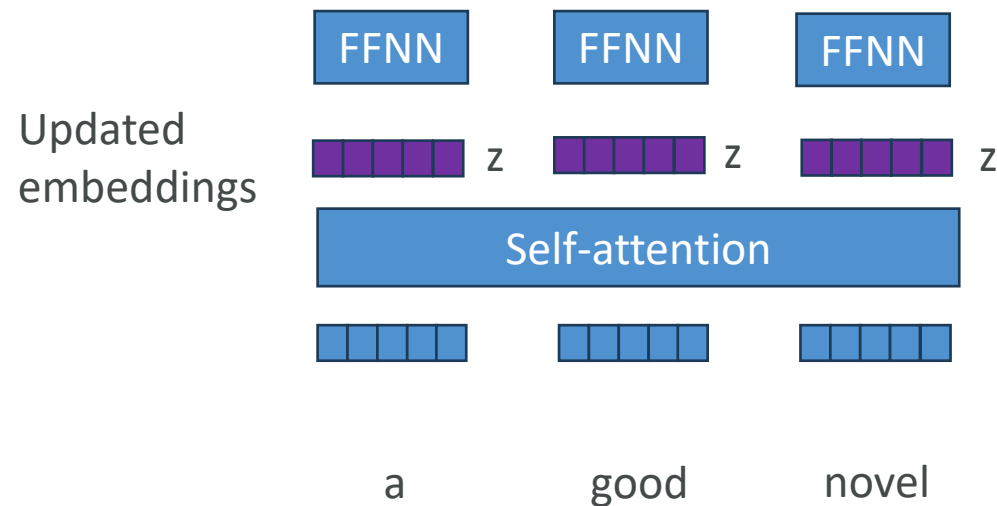
Self-Attention (in more depth)

- Now we sum.
- Multiply values by scores, and sum them up.



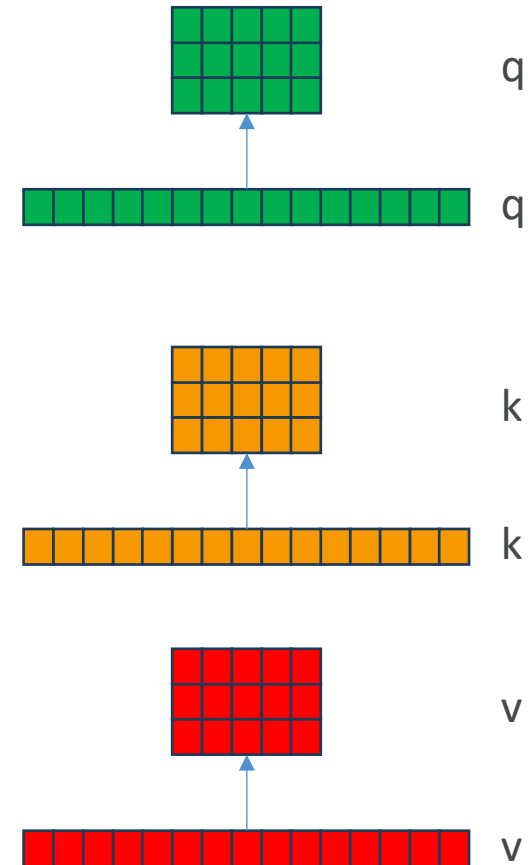
Self-Attention (in more depth)

- Repeat entire process for each token (in parallel)
- Now we have our updated embeddings for each token!
- These weight each token embedding as it's passed into the feed-forward neural network.



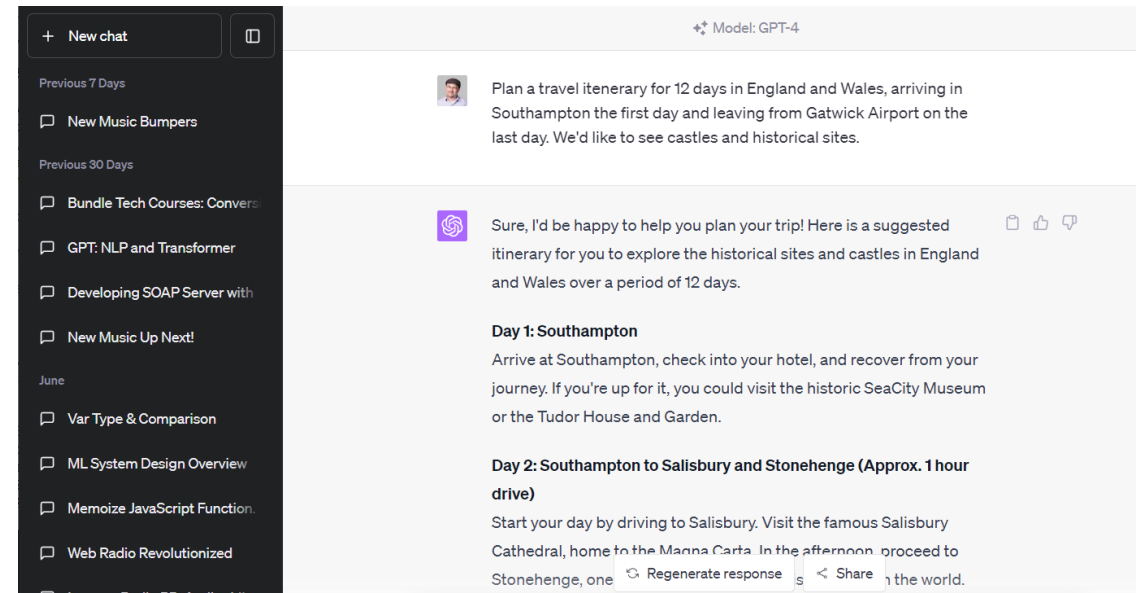
Multi-Headed Self-Attention

- The q , k , and v vectors are reshaped into matrices
- Then each row of the matrix can be processed in parallel
- The number of rows are the number of “heads”



Applications of Transformers

- Chat!
- Question answering
- Text classification
 - i.e., sentiment analysis
- Named entity recognition
- Summarization
- Translation
- Code generation
- Text generation
 - i.e., automated customer service

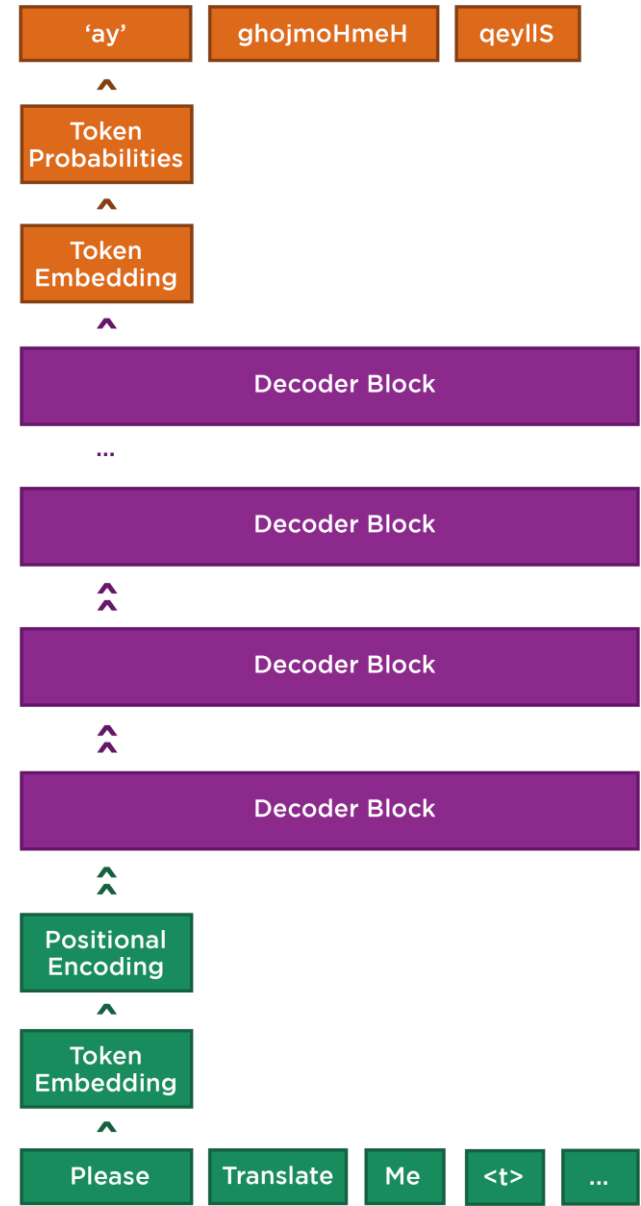
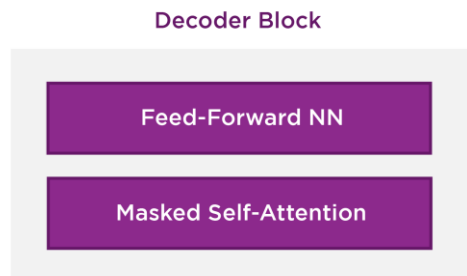


From Transformers to LLM's

Large Language Models

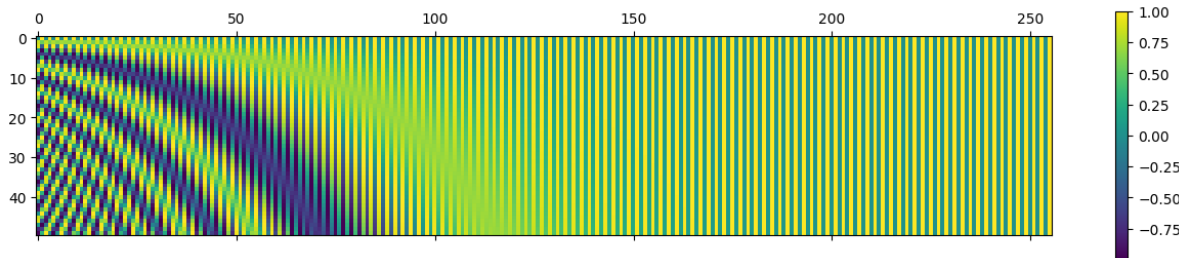
GPT architecture

- Generative Pre-Trained Transformer (GPT-2 in this example)
 - Other LLM's are similar
- Decoder-only – stacks of decoder blocks
 - Each consisting of a masked self-attention layer, and a feed-forward neural network
 - As an aside, BERT consists only of encoders. T5 is an example of a model that uses both encoders and decoders.
- No concept of input, all it does is generate the next token over and over
 - Using attention to maintain relationships to previous words / tokens
 - You “prompt” it with the tokens of your question or whatever
 - It then keeps on generating given the previous tokens
- Getting rid of the idea of inputs and outputs is what allows us to train it on unlabeled piles of text
 - It's “learning a language” rather than optimizing for some specific task
- Hundreds of billions of parameters



LLM Input processing

- Tokenization, token encoding
- Token embedding
 - Captures semantic relationships between tokens, token similarities
- Positional encoding
 - Captures the position of the token in the input relative to other nearby tokens
 - Uses an interleaved sinusoidal function so it works on any length



GPT-3 Codex

Many words map to one token, but some don't: indivisible.

Unicode characters like emojis may be split into many tokens containing the underlying bytes: 🍌

Sequences of characters commonly found next to each other may be grouped together: 1234567890

Clear

Show example

Tokens

64

Characters

252

Many words map to one token, but some don't: indivisible.

Unicode characters like emojis may be split into many tokens containing the underlying bytes: 🍌🍌🍌🍌🍌

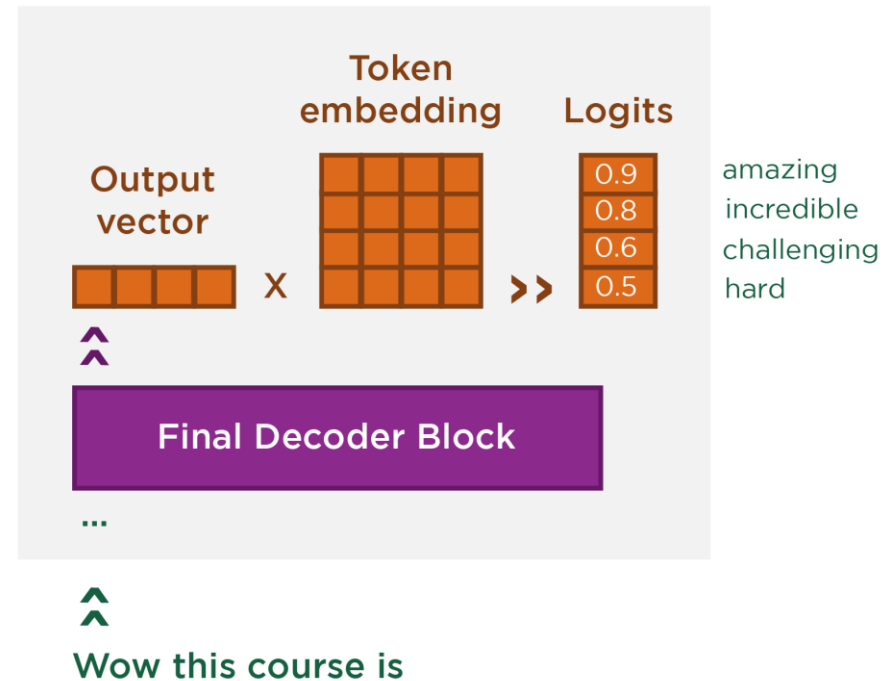
Sequences of characters commonly found next to each other may be grouped together: 1234567890

TEXT

TOKEN IDS

LLM Output processing

- The stack of decoders outputs a vector at the end
- Multiply this with the token embeddings
- This gives you probabilities (logits) of each token being the right next token (word) in the sequence
- You can randomize things a bit here (“temperature”) instead of always picking the highest probability



LLMs: Key Terms

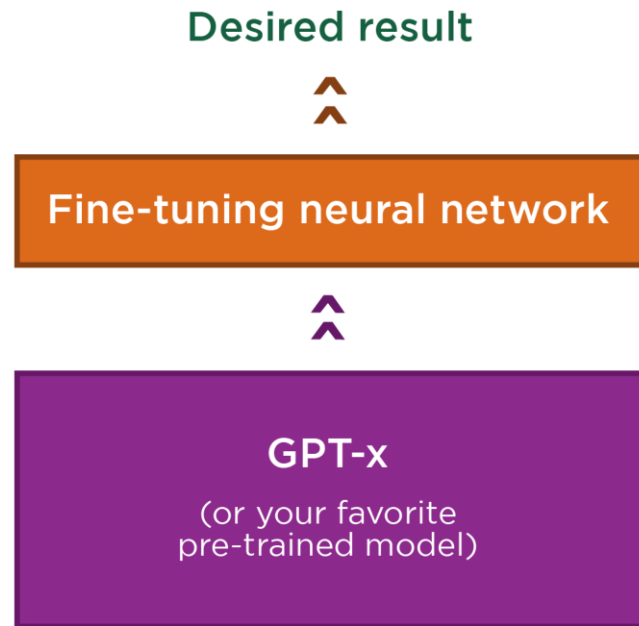
- **Tokens** – numerical representations of words or parts of words
- **Embeddings** – mathematical representations (vectors) that encode the “meaning” of a token
- **Top P** – Threshold probability for token inclusion (higher = more random)
- **Top K** – Alternate mechanism where K candidates exist for token inclusion (higher = more random)
- **Temperature** – the level of randomness in selecting the next word in the output from those tokens
 - High temperature: More random
 - Low temperature: More consistent

Context window – The number of tokens an LLM can process at once

Max tokens – Limit for total number of tokens (on input or output)

Transfer Learning (Fine Tuning) with Transformers

- Add additional training data through the whole thing
- Freeze specific layers, re-train others
 - Train a new tokenizer to learn a new language
- Add a layer on top of the pre-trained model
 - Just a few may be all that's needed!
 - Provide examples of prompts and desired completions
 - “How’s the weather?” -> “What’s it to you, bucko?”
 - Adapt it to classification or other tasks
 - “Wow, I love this course!” -> “Positive”



Generative AI in AWS

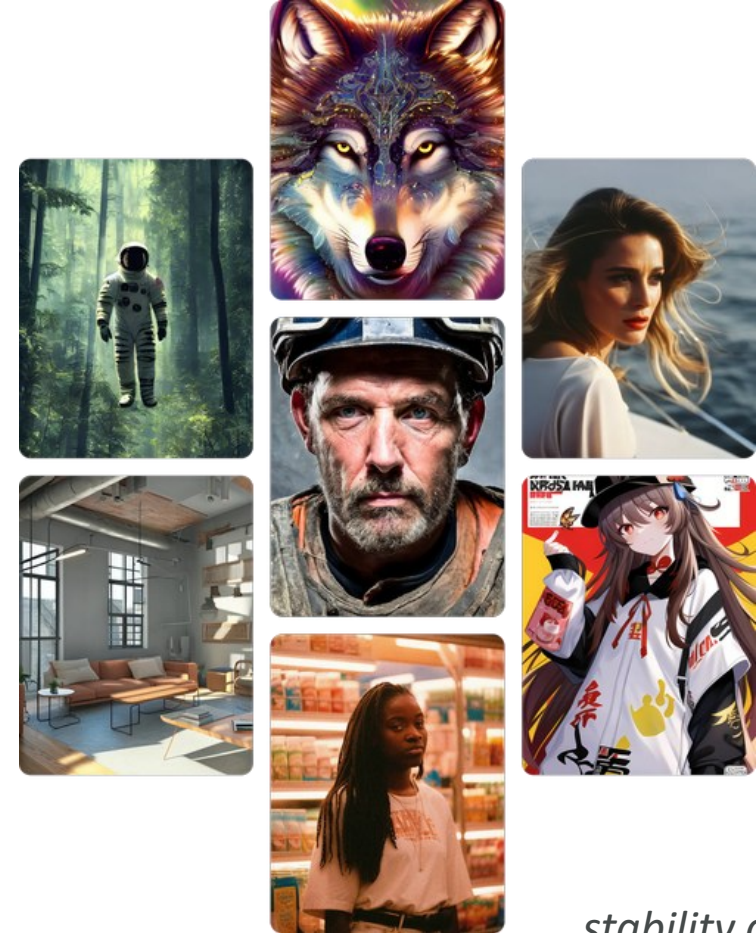
Foundation Models

- The giant, pre-trained transformer models we are fine tuning for specific tasks, or applying to new applications
- GPT-n (OpenAI, Microsoft)
- Claude (Anthropic)
- BERT (Google)
- DALL-E (OpenAI, Microsoft)
- LLaMa (Meta)
- Segment Anything (Meta)
- Where's Amazon?



AWS Foundation Models (Base Models)

- Jurassic-2 (AI21labs)
 - Multilingual LLMs for text generation
 - Spanish, French, German, Portuguese, Italian, Dutch
- Claude (Anthropic)
 - LLM's for conversations
 - Question answering
 - Workflow automation
- Stable Diffusion (stability.ai)
 - Image, art, logo, design generation
- Llama (Meta)
 - LLM
- Amazon Titan
 - Text summarization
 - Text generation
 - Q&A
 - Embeddings
 - Personalization
 - Search



stability.ai

Amazon SageMaker Jumpstart with Generative AI

- SageMaker Studio has a “JumpStart” feature
- Lets you quickly open up a notebook with a given model loaded up and ready to go
- Current foundation models
 - Hugging Face models (text generation)
 - Falcon, Flan, BloomZ, GPT-J
 - Stable Diffusion (image generation)
 - Amazon Alexa (encoder/decoder multilingual LLM)

